

POS System

UI/UX & Functional Design Specification

Master design reference for Categories, Products, Inventory, Customers, Sales, Purchases and Invoices.

Version 1.0

Prepared from the supplied POS component source

POS System — UI/UX & Functional Design Specification

Version: 1.0 **Source:** Supplied components .zip **Audience:** UI/UX designers, frontend developers, backend developers, QA engineers and product owners **Purpose:** This document defines the common design rules and module-by-module behavior required to reproduce the POS application consistently.

1. How to Use This Document

A designer should be able to create the complete module without opening the source code.

For every module, follow this order:

1. Build the page shell.
2. Build the list/table.
3. Build search and filters.
4. Build Add/Create form.
5. Reuse the same form for Edit.
6. Add View/Details where specified.
7. Add Delete/Archive confirmation.
8. Implement loading, empty, error and success states.
9. Implement responsive behavior.
10. Verify validation and permissions.

Important: Frontend validation improves UX, but the backend must always validate the same rules again.

2. Global Page Structure

Every master-data page should follow this structure:

Breadcrumb		
Page Title	[Primary Action]	
Short description		
Search / Filters / View Controls		
Optional KPI / Summary Cards		
Data Table		
Pagination / Result Count		

Page header

Required:

- Page title
- Primary action
- Optional description

Examples:

- Categories $\rightarrow$ + Add Category
- Products $\rightarrow$ + Add Product

- Customers → + Add Customer
- Invoices → + Create Invoice or View Sales

3. Standard CRUD Pattern

Create

Use a modal/dialog for short forms.

Use a dedicated page for complex forms such as:

- New Sale
- Purchase Order
- Invoice Designer

Buttons:

[Cancel] [Create / Save]

The primary button must be visually dominant.

Edit

Use the same fields as Create unless a field is intentionally immutable.

Button:

[Cancel] [Save Changes]

View

Use a detail page or large dialog.

Do not make read-only details look like editable inputs.

Delete

Never delete immediately.

Use:

Delete [Entity]

Are you sure you want to delete "Example"?

This action cannot be undone.

[Cancel] [Delete]

For financial or historical records, use **Cancel / Void / Archive** instead of hard delete.

4. Form Standards

Field anatomy

Each field should contain:

Field Label *
[Input / Select / Date / Textarea]

Helper text (optional)
Error message (only when invalid)

Required fields

Mark with *.

Optional fields

Do not add *.

Read-only fields

Use a muted background and prevent editing.

Calculated fields

Clearly indicate that the value is calculated.

Example:

Total
£1,250.00
Calculated from line items, tax and discount

5. Input Types

Data	Recommended control
Name	Text input
Email	Email input
Phone	Tel input
Currency	Numeric input with 2 decimals
Quantity	Numeric input
Percentage	Numeric input with 0–100
Date	Date picker
Status	Select
Category	Searchable select
Description	Textarea
Long notes	Textarea
Image	Drag/drop upload
Boolean	Switch/checkbox
Multiple selection	Multi-select
Product search	Search/command selector

6. Standard Validation

Field type	Rule
Name	Required where specified; trim whitespace
Email	Valid email format
Phone	Valid phone format
Currency	>= 0 unless explicitly allowing negative
Percentage	0–100
Quantity	> 0 for transaction quantities
Stock	>= 0 by default
Date	Valid date

Field type	Rule
File	Validate MIME type and size
ID	Must exist and belong to the allowed scope

Errors should appear beside the field.

Example:

```
Retail Price *
[ -10.00 ]

Retail price cannot be negative.
```

7. Standard Table Pattern

Tables should support:

- Header
- Row hover
- Sorting where useful
- Search
- Filters
- Pagination
- Empty state
- Loading state
- Row actions

Standard action order:

```
View → Edit → Duplicate/Other → Delete
```

Use icons with tooltips, but important destructive actions should still be understandable without relying on the icon alone.

8. Standard States

Every module must define:

Loading

Show skeleton/table placeholders.

Empty

Example:

No products found.

Add your first product to get started.

Error

Example:

Unable to load products. Please try again.

Button:

[Ret ry]

Success

Use a toast/banner:

Product created successfully.

Validation failure

Keep the form open and highlight invalid fields.

Delete confirmation

Never close the underlying page until confirmation is completed.

9. Responsive Rules

Desktop

- Use full-width tables.
- Forms can use 2-column layouts.
- Keep primary actions in the page header.

Tablet

- Reduce table columns.
- Move less important columns into row details.

Mobile

- Convert tables into cards or horizontally scrollable tables.
- Forms become one column.
- Keep primary action accessible.
- Avoid tiny icon-only actions for critical operations.

10. Permissions

Recommended permission structure:

Action	Permission
View	<code>module.view</code>
Create	<code>module.create</code>
Edit	<code>module.edit</code>
Delete	<code>module.delete</code>
Export	<code>module.export</code>
Manage payments	<code>module.payment</code>
Manage stock	<code>inventory.adjust</code>

The UI should hide unavailable actions, but the backend must enforce permissions.

11. Financial Rules

Never trust totals calculated by the browser.

For:

- Sales
- Purchases
- Invoices
- Payments
- Customer credit

the server must recalculate:

```
Subtotal
Discount
Tax
Shipping
Grand Total
Amount Paid
Amount Due
```

All stock/payment/invoice operations should use database transactions.

12. Design Consistency Rules

Use the same:

- Button hierarchy
- Dialog width
- Label spacing
- Input height
- Currency format
- Date format
- Status badge style
- Toast behavior
- Confirmation dialog
- Empty-state pattern

A user should feel that every module belongs to the same application.

Module Specification — Product Categories

1. Purpose

Manage the product category master.

2. Page

Route suggestion: /categories

Header

Title: Categories **Primary action:** + Add Category

List

Field	Display
Category Name	Text
Product Count	Number
Status	Badge, if enabled
Created At	Date
Actions	Edit / Delete

3. Create Form

Form title

Add Category

Field	Control	Required	Validation
Category Name	Text	Yes	Non-empty, trimmed, unique
Status	Select	Optional	Active / Inactive

Placeholder

Enter category name

Actions

Cancel / Add Category

4. Edit

Title:

Edit Category

Prefill all existing values.

Action:

Save Changes

5. Delete

Delete is allowed only when the category has no dependent products, or the system must provide reassignment/archive behavior.

Confirmation:

Delete Category

Deleting this category may affect products assigned to it.

6. Search/Filtering

If the category count grows:

- Search by category name
- Filter by status

7. States

- Loading
- Empty
- Search empty
- Create success
- Update success
- Delete success
- Duplicate category error
- API error

8. Designer checklist

- Page header
- Add Category button
- Category table
- Add dialog
- Edit dialog
- Delete confirmation
- Validation/error states
- Empty state
- Mobile layout

Module Specification — Products

1. Purpose

Products are the central master records used by inventory, sales, purchases and invoices.

2. Product List

Route suggestion: /products

Header

Title: Products **Primary action:** + Add Product

Search

Placeholder:

Search product, SKU or category...

Filters

- Category
- Stock status

Stock statuses:

- OK
- Low
- Out of Stock

Table columns

Column	Type
Select	Checkbox
Image	Thumbnail
Product Name	Text
SKU	Text
Category	Text
Wholesale	Currency
Trader	Currency
Retail	Currency
Stock	Number
Value	Currency
Variants	Count/badge
Actions	Buttons

Allow optional column visibility controls.

3. Add Product Form

Section A — Product Image

Field	Type	Required
Product Image	Drag/drop upload	No
Image URL	URL/text	No

Supported source formats:

- PNG
- JPG/JPEG
- WEBP

Source UI indicates a maximum image size of 5 MB.

Features:

- Preview
- Remove
- Replace
- Drag/drop
- Image URL

Section B — Product Information

Field	Control	Required	Placeholder
Product Name	Text	Yes	Product name
SKU	Text	No	Auto-generated if blank
Category	Searchable Select	Yes	Select Category
Wholesale Price	Currency	No	0.00
Trader Price	Currency	No	0.00
Retail Price	Currency	Yes	0.00
Stock	Number	Yes	0

Validation

- Product Name: required.
- SKU: unique if supplied.
- Category: required.
- Prices: ≥ 0 .
- Stock: ≥ 0 .

4. Edit Product

Same form as Add Product.

Title:

Edit Product

Action:

Save Changes

5. Product Actions

Each row may contain:

- Edit
- Manage Variants
- Duplicate
- Delete

Duplicate

Duplicate all safe master fields but generate a new unique SKU.

Delete

If historical transactions reference the product, prefer archive/inactive.

6. Bulk Management

The supplied UI supports bulk operations.

Bulk Price

Fields:

Field	Control
Apply To	Selected products
Operation	Select
Value	Number

Operations:

- Increase
- Decrease
- Set
- Percentage
- Fixed amount

Bulk Stock

Field	Control
Apply To	Selected products
Operation	Select
Quantity	Number

Bulk Category

Allow:

- Existing category
- New category

Bulk SKU

Field:

SKU Pattern

Example:

{CAT} - {NAME} - {NUM}

Always show an operation preview before applying bulk changes.

7. Variants

If variants exist, show:

Manage Variants

Recommended variant fields:

- Variant name
- SKU
- Price
- Stock
- Attributes
- Status

Variants must have unique identifiers.

8. Designer Checklist

- Product list
- Search
- Category filter
- Stock filter
- Column visibility
- Add form
- Edit form
- Image upload
- Image preview
- Variant action
- Duplicate
- Delete confirmation
- Bulk management
- Mobile responsive design

Module Specification — Inventory

1. Purpose

Display and maintain current stock quantities.

Route suggestion: /inventory

2. Inventory List

Search

Placeholder:

Search inventory...

Recommended filters

- Warehouse
- Category
- Stock status

Table

Column	Display
Product	Name
SKU	Text
Category	Text
Current Stock	Number
Reorder Level	Number
Status	Badge
Stock Value	Currency
Actions	Update/View

3. Stock Status

Recommended rules:

```
stock <= 0  
→ OUT OF STOCK
```

```
stock > 0 AND stock <= reorder_level  
→ LOW STOCK
```

```
stock > reorder_level  
→ IN STOCK
```

4. Update Stock

Dialog title

Update Stock

Field	Control	Required
Product	Read-only	Yes

Field	Control	Required
Current Stock	Read-only	Yes
New Stock Quantity	Number	Yes
Reason	Textarea	Recommended

Placeholder:

Enter new stock quantity

Actions

Cancel / Update Stock

5. Stock Audit

Every adjustment should create a stock movement.

Recommended record:

```
{
  "product_id": 77,
  "warehouse_id": 1,
  "before": 20,
  "after": 25,
  "change": 5,
  "reason": "Physical count",
  "performed_by": 1
}
```

Never silently overwrite stock without an audit trail.

6. Rules

- Stock cannot be negative unless explicitly enabled.
- Stock updates must be atomic.
- Warehouse stock must be independent where warehouses are supported.
- Sales decrease stock.
- Purchase receiving increases stock.
- Returns adjust stock according to return type.

7. Designer Checklist

- Inventory search
- Filters
- Stock status badges
- Stock table
- Update stock dialog
- Reason field
- Audit/movement view
- Loading state
- Empty state

Module Specification — Customers

1. Purpose

Manage customers, pricing type, contact information and credit activity.

Route suggestion: /customers

2. Customer List

Search

Placeholder:

Search customers...

Search:

- Name
- Phone
- Email

Filters

- Customer Type
- Status
- Credit status

Table

Column	Type
Customer	Name
Type	Badge
Phone	Text
Email	Text
Credit Limit	Currency
Current Balance	Currency
Status	Badge
Actions	View / Edit / Delete

3. Customer Form

Field	Control	Required	Placeholder
Customer Name	Text	Yes	Enter customer name
Customer Type	Select	Yes	Select type
Phone Number	Tel	Yes	+1 (555) 123-4567
Email	Email	No	customer@example.com
Address	Textarea	No	Enter customer address
Credit Limit	Currency	No	1000.00

Field	Control	Required	Placeholder
Current Balance	Read-only	No	Calculated

Customer Types

The supplied application uses:

- Wholesaler
- Trader
- Retailer

The pricing/discount rules should be configurable.

4. Customer Detail

Sections:

1. Profile
2. Orders
3. Invoices
4. Payments
5. Credit Ledger

5. Credit Transaction

Form

Field	Control	Required
Amount	Currency	Yes
Description	Textarea	Yes
Reference	Text	No

Placeholder:

Enter reason for credit transaction

Reference example:

Invoice number, receipt number, etc.

6. Customer Orders

Show:

- Invoice/order number
- Date
- Payment method
- Status
- Total
- Items purchased

Provide filters for:

- Customer
- Payment method
- Status
- Invoice date

7. Delete

If historical transactions exist:

Do not hard delete.

Use:

- Archive
- Deactivate

8. Designer Checklist

- Customer list
- Search
- Filters
- Add customer
- Edit customer
- Customer details
- Credit transaction
- Order history
- Ledger
- Delete/archive confirmation
- Mobile layout

Module Specification — Sales / New Sale

1. Purpose

Create sales transactions and generate invoices.

The source contains both a sales workflow and a detailed New Sale screen.

Route suggestion: /sales/new

2. Screen Layout

Recommended:

■ New Sale

■ Product Search

■ Customer / Invoice Information

■ Product Line Items

■ Summary

■ Subtotal

■ Discount

■ Tax

■ Shipping

■ Grand Total

[Save Draft]

■

■

■

■

■ Payment

■ Receive Amount

■ Payment Type

■ Change

■ Due

■ Notes

3. Product Search

Placeholder:

Search by name, SKU or category...

Support:

- Product name
- SKU
- Category
- Category filter

Selecting a product adds a line item.

4. Invoice Information

Field	Control	Required
Customer	Searchable Select	Optional
Invoice Number	Text	Optional/system-generated
Invoice Date	Date	Yes
Due Date	Date	Conditional

5. Customer

Allow:

- Existing customer selection
- Walk-in customer
- Add Customer

Quick Add Customer

Field	Required
Full Name	Yes
Customer Type	Yes
Phone	Yes
Email	No
Address	No
Credit Limit	No

6. Sale Line Item

Field	Type	Required
Product	Selected product	Yes
SKU/Code	Read-only	Yes
Unit	Read-only	Yes
Quantity	Number	Yes
Sale Price	Currency	Yes
Discount	Percentage	No
VAT/Tax Type	Select	Yes
VAT/Tax Rate	Percentage/select	No
Tax Amount	Calculated	Read-only
Line Total	Calculated	Read-only
Remove	Action	—

VAT rates present in source UI

- 0%
- 5%
- 10%
- 12%
- 18%
- 20%

These should be configurable.

7. Custom Product

The source supports adding a custom product.

Fields:

Field	Required
Description	Yes
Price	Yes
Quantity	Yes
VAT/Tax Rate	No
Discount	No

Example description:

Custom Brake Pad Set – Toyota Corolla

8. Sale Totals

Display:

Subtotal
Discount
Tax
Shipping Charge
Grand Total

Recommended calculation:

Line subtotal = quantity × unit price

Discount = line subtotal × discount%

Taxable amount = line subtotal - discount

Tax = taxable amount × tax%

Grand total = subtotal - discount + tax + shipping

The server must recalculate all totals.

9. Payment

Fields:

Field	Control
Receive Amount	Currency input
Change Amount	Calculated
Due Amount	Calculated
Payment Type	Select
Shipping Charge	Currency
Note	Textarea

Payment methods:

- Cash
- Card
- Bank Transfer
- Cheque
- Other

10. Draft

Save:

- Customer
- Invoice data
- Items
- Prices
- Discount
- Tax
- Shipping
- Payment
- Notes

Draft creation must not reduce stock.

11. Complete Sale

Before completion:

1. Validate customer.
2. Validate products.
3. Validate quantities.
4. Validate stock.
5. Recalculate prices/tax/discount.
6. Save sale.
7. Create invoice.
8. Create payment.
9. Reduce stock.
10. Commit transaction.

If any critical step fails, rollback.

12. Designer Checklist

- Product search
- Category filter
- Customer selection
- Add customer
- Invoice metadata
- Line-item table
- Quantity editing
- Price editing
- Discount
- VAT/tax

- Custom product
- Shipping
- Payment
- Change/due calculation
- Save draft
- Complete sale
- Confirmation
- Responsive layout

Module Specification — Purchases

1. Purpose

Manage suppliers, purchase orders and purchase receiving.

Route suggestion: /purchases

2. Supplier Management

Supplier list

Search:

Search suppliers...

Recommended columns:

- Supplier Name
- Company
- Email
- Phone
- City/State
- Tax Number
- Opening Balance
- Payment Terms
- Lead Time
- Status
- Actions

3. Add/Edit Supplier Form

Field	Control	Required
Supplier Name	Text	Yes
Company Name	Text	No
Email	Email	Yes in supplied UI
Phone	Tel	Yes
Address	Textarea	No
City	Text	No
State	Text	No
Country	Text	No
Postal Code	Text	No
GST / Tax Number	Text	No
PAN Number	Text	No
Opening Balance	Currency	No
Payment Terms	Select	No

Field	Control	Required
Lead Time (days)	Number	No
Category	Text/select	No
Status	Select	Yes
Notes	Textarea	No

Status:

- Active
- Inactive
- Blacklisted

4. Purchase Order

Header

Field	Control	Required
Supplier	Searchable Select	Yes
Expected Date	Date	No
Order Status	Select	Yes
Notes	Textarea	No

Statuses:

- Draft
- Sent
- Confirmed
- Cancelled

5. Purchase Items

Field	Control	Required
Product	Searchable Select	Yes
Quantity	Number	Yes
Cost Price	Currency	Yes
Remove	Action	—

Calculated:

Item Total = Quantity × Cost Price

Purchase Total = Sum(Item Total)

6. Receiving

Receiving is separate from ordinary purchase editing.

Receive fields

Field	Type
Product	Read-only
Ordered Quantity	Read-only
Previously Received	Read-only
Total Received	Number

Example:

Ordered: 100
Previously Received: 40
Total Received: 70

New stock received = 30

Do not add 70 to stock again.

7. Purchase Stock Rules

Status	Stock effect
Draft	None
Sent	None
Confirmed	None
Partially Received	Add newly received amount
Received	Add newly received amount
Cancelled	None

8. Designer Checklist

- Supplier list
- Supplier search
- Supplier filters
- Add supplier
- Edit supplier
- Purchase order list
- Create purchase order
- Product selection
- Quantity
- Cost price
- Status
- Receive purchase
- Partial receive
- Receiving confirmation
- Document view/share actions
- Mobile layout

Module Specification — Invoices

1. Purpose

View, manage and collect payments against invoices.

Route suggestion: /invoices

2. Invoice List

Search

Placeholder:

Search by invoice ID or customer...

Status filter

Support:

- All
- Paid
- Pending
- Overdue
- Draft
- Sent
- Partially Paid
- Cancelled

Table

Column	Display
Invoice Number	Text
Customer	Text
Issue Date	Date
Due Date	Date
Subtotal	Currency
Tax	Currency
Total	Currency
Paid	Currency
Due	Currency
Status	Badge
Actions	Buttons

3. Invoice Actions

Recommended:

- View Details
- Invoice Designer

- Email Invoice
- Mark as Sent
- Record Payment
- Print
- Download
- Cancel/Void

4. Invoice Details

Header

Display:

- Invoice number
- Issue date
- Due date
- Status

Customer

Display:

- Customer name
- Type
- Phone
- Email
- Address

Items

Column	Type
Description	Text
Quantity	Number
Unit Price	Currency
Amount	Currency

Totals

Subtotal
Tax
Total
Paid
Amount Due

5. Record Payment

Dialog

Title:

Record Payment

Field	Control	Required
Invoice Total	Read-only currency	—
Remaining Due	Read-only currency	—
Amount	Currency	Yes
Payment Method	Select	Yes
Reference	Text	No
Notes	Textarea	No

Payment method options may include:

- Cash
- Card
- Bank Transfer
- Credit
- Cheque
- Mixed

Source placeholders:

Receipt / transaction reference

Payment notes

6. Payment Validation

Amount > 0
Amount <= Remaining Due

Unless overpayment is an explicit supported business case.

7. Payment History

Field	Display
Date/Time	Date/time
Amount	Currency
Method	Badge
Reference	Text
Notes	Text
Recorded By	User

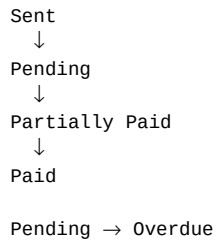
Financial history should be immutable.

Corrections should use reversal/adjustment transactions.

8. Invoice Status

Recommended lifecycle:

Draft
↓



Cancellation should only be possible where business rules allow it.

9. Delete Rule

Finalized invoices must not be hard deleted.

Use:

Cancel / Void

with:

- reason
- user
- timestamp
- audit trail

Draft invoices may be deleted.

10. Designer Checklist

- Invoice list
- Search
- Status filter
- Invoice details
- Details tab
- Payments tab
- Record payment dialog
- Status control
- Email action
- Invoice designer
- Print/download
- Cancel/void confirmation
- Responsive design

POS Design System — Reusable Components

This document defines the reusable visual components that should be created before designing individual modules.

1. Buttons

Primary

Use for:

- Add
- Create
- Save
- Complete sale
- Confirm payment

Examples:

- Add Product
- Create Purchase
- Save Changes

Secondary

Use for:

- Cancel
- Back
- Filter
- Close

Destructive

Use only for:

- Delete
- Cancel invoice
- Void payment

Always require confirmation for destructive operations.

2. Status Badges

Use consistent colors and wording.

Status type	Example
Success	Paid / Active / In Stock
Warning	Pending / Low Stock
Error	Overdue / Out of Stock / Cancelled
Neutral	Draft
Info	Sent / Confirmed

Do not rely on color alone; always include text.

3. Dialog Sizes

Recommended:

Dialog	Width
Simple delete	Small
Category	Small
Customer	Medium
Product	Large
Supplier	Large
Payment	Medium
Product variants	Large
Complex transaction	Extra large

4. Spacing

Use a consistent 4/8px spacing scale.

Recommended:

- Label → input: 6–8px
- Field → field: 16–20px
- Form section → section: 24px
- Page section → section: 32px

5. Currency

Use one currency format across the application.

Recommended:

£1,250.00

If another market is used, currency should be configurable.

Never mix currency symbols in the same screen.

6. Date

Use one application-wide date format.

Recommended display:

14 Aug 2026

Date inputs should use the native/application date picker.

7. Toast Messages

Success

Product created successfully.

Error

Unable to save product. Please try again.

Warning

Only 5 units are available.

Delete

Product deleted successfully.

8. Confirmation Dialogs

Every destructive dialog should explain the consequence.

Example:

```
Delete Product

Are you sure you want to delete "Air Filter"?

Products with historical sales should be archived instead.

[Cancel] [Delete]
```

9. Search

Search fields should:

- Debounce API calls where appropriate.
- Show a clear button.
- Preserve search during pagination.
- Show No results found separately from an empty database.

10. Forms

Complex forms should be grouped into sections.

Example Product:

```
Product
  ■■■ Image
  ■■■ Basic Information
  ■■■ Pricing
  ■■■ Inventory
  ■■■ Variants
```

Example Sale:

```
Sale
  ■■■ Customer
  ■■■ Invoice Information
  ■■■ Products
  ■■■ Totals
  ■■■ Payment
  ■■■ Notes
```

11. Accessibility

Every design must provide:

- Visible labels
- Keyboard focus
- Accessible error messages
- Sufficient contrast
- Tooltips for icon-only controls
- Confirmation for destructive actions
- Screen-reader-friendly buttons

12. Mobile

On mobile:

- Forms become single-column.
- Tables become cards or scroll containers.
- Primary action stays visible.
- Filters can move into a filter drawer.
- Dialogs may become full-screen sheets.

POS API & Data Rules

This is the implementation contract that should accompany the UI designs.

1. Categories

List

GET /categories

Create

POST /categories

```
{
  "name": "Air Filters",
  "status": "active"
}
```

Update

PUT /categories/{id}

Delete

DELETE /categories/{id}

2. Products

List

GET /products?search=&category_id=&stock_status=

Create

POST /products

```
{
  "name": "Air Filter",
  "sku": "AF-001",
  "category_id": 2,
  "wholesale_price": 20,
  "trader_price": 23,
  "retail_price": 25,
  "stock": 100
}
```

Update

PUT /products/{id}

Delete

DELETE /products/{id}

Bulk

POST /products/bulk-update

3. Inventory

List

GET /inventory

Update stock

PATCH /inventory/{product_id}/stock

```
{
  "quantity": 25,
  "reason": "Physical stock count"
}
```

Movements

GET /inventory/{product_id}/movements

4. Customers

List

GET /customers?search=&type=

Create

POST /customers

Update

PUT /customers/{id}

Delete/archive

DELETE /customers/{id}

Ledger

GET /customers/{id}/ledger

Credit transaction

POST /customers/{id}/credit-transactions

5. Sales

List

GET /sales

Draft

POST /sales/drafts

Update draft

PUT /sales/drafts/{id}

Complete sale

POST /sales

The server must:

1. Validate product IDs.
2. Validate stock.
3. Validate customer.
4. Recalculate prices.
5. Recalculate tax.
6. Recalculate discounts.
7. Create sale.

8. Create invoice.
9. Create payment if applicable.
10. Reduce inventory.
11. Commit transaction.

6. Purchases

Suppliers

GET /suppliers
POST /suppliers
PUT /suppliers/{id}

Purchase orders

GET /purchase-orders
POST /purchase-orders
PUT /purchase-orders/{id}

Receive

POST /purchase-orders/{id}/receive

Partial receive

POST /purchase-orders/{id}/partial-receive

Receiving must add only the newly received quantity.

7. Invoices

List

GET /invoices?search=&status=

Detail

GET /invoices/{id}

Status

PATCH /invoices/{id}/status

Payment

POST /invoices/{id}/payments

```
{
  "amount": 250,
  "method": "bank_transfer",
  "reference": "TXN-12345",
  "notes": "Customer payment"
}
```

8. API Error Format

Use one consistent structure:

```
{
  "message": "Validation failed.",
  "errors": {
```

```
    "quantity": [  
      "Quantity must be greater than zero."  
    ]  
  }  
}
```

9. Authorization

Every API must verify:

- Authenticated user
- Role
- Permission
- Tenant/store/warehouse scope where applicable

Never rely on the frontend to enforce permissions.

10. Transactions

Database transactions are required for:

- Completing sales
- Receiving purchases
- Recording invoice payments
- Stock adjustments
- Credit ledger changes

If one part fails, rollback all dependent changes.

Designer Handoff Checklist

Use this checklist whenever a new POS module is designed.

Page

- Page title
- Breadcrumb
- Page description
- Primary action
- Search
- Filters
- Result count
- Table/card layout

Create/Edit

- Form title
- Every field has a label
- Required fields marked
- Placeholder/helper text
- Validation message
- Save button
- Cancel button
- Loading state
- Server error state

List

- Loading state
- Empty state
- Search-empty state
- Pagination
- Row actions
- Delete confirmation
- Responsive behavior

Business logic

- Required relationships identified
- Calculated values identified
- Stock effect identified
- Payment effect identified
- Status transitions identified
- Audit requirements identified
- Permission requirements identified

Mobile

- 320px minimum consideration
- Single-column forms
- Horizontal table scroll/card conversion
- Full-screen complex dialogs
- Touch-friendly controls

QA

- Create valid record
- Create invalid record
- Edit record
- Delete/archive record
- Search
- Filter
- Pagination
- Permission denied
- API failure
- Empty database
- Slow network

Source Component Map

The following non-UI source components were found in the supplied ZIP and can be mapped to the specification.

- components/advanced-search.tsx
- components/barcode-scanner.tsx
- components/categories-view.tsx
- components/customer-ledger-view.tsx
- components/customer-orders-view.tsx
- components/customer-portal-nav.tsx
- components/customer-portal.tsx
- components/customers-view.tsx
- components/dashboard-view.tsx
- components/document-capture-dialog.tsx
- components/email-invoice-dialog.tsx
- components/inventory-view.tsx
- components/invoice-document.tsx
- components/invoice-template-settings.tsx
- components/invoices-view.tsx
- components/layout.tsx
- components/new-sales-view.tsx
- components/not-found.tsx
- components/portal-admin-panel.tsx
- components/price-management-view.tsx
- components/product-management-view.tsx
- components/product-shelving-view.tsx
- components/protected-route.tsx
- components/purchase-management-view.tsx
- components/reports-view.tsx
- components/return-management-view.tsx
- components/sales-view.tsx
- components/send-share-dialog.tsx
- components/staff-management-view.tsx
- components/staff-portal.tsx
- components/storefront-account-page.tsx
- components/storefront-brand-page.tsx
- components/storefront-checkout-page.tsx
- components/storefront-home-page.tsx
- components/storefront-product-detail-page.tsx
- components/storefront-products-page.tsx

- components/storefront-static-pages.tsx
- components/storefront-view.tsx
- components/unified-portal.tsx
- components/variant-management-dialog.tsx
- components/warehouse-management-view.tsx
- components/website-management-view.tsx
- components/website-orders-tab.tsx
- components/xero-invoices-view.tsx